

МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №7
по дисциплине «Операционные системы»
Тема: Профилирование (анализ динамики выполнения и событий)

Студентка гр. 2304

Иванова М.А.

Преподаватель

Душутина Е.В.

Санкт-Петербург

2024

Цель работы.

Изучить профилирование, способы трассировки ядра, применить их на практике и проанализировать динамику выполнения и событий.

Задание.

Порождение процессов, их переключение (смена контекста) и конкуренция.

Выполнение работы.

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~$ neofetch
      ...-:::-...
    .-MMMMMMMMMMMMMMM-.
   .-MMMM`..:::-...`MMMM-.
  .:MMM.:MMMMMMMMMMMMMM.:MMM:.
 -MMM-M--MMMMMMMMMMMMMMMMMMM.MMM-
`:MMM:MM` :MMM:.....-MM:MM:`
:MMM:MMM` :MM: ```` `:MM:MM:
MMM.MMMM` :MM. -MM. .MM- `MMMM.MMM.
:MMM:MMM` :MM. -MM- .MM: `MMMM-MMM:
:MMM:MMM` :MM. -MM- .MM: `MMMM:Mmm:
:MMM:MMM` :MM: --MM:--MM: `MMMM.MMM.
:MMM:MMM- `MMMMMMMMMMMMMM-` -MMM-MMM:
:MMM:MMM:`           `:MMM:MMm:
 .MMM.MMMM:-----:MMM.MMM.
 '-MMM.-MMMMMMMMMMMMMM-.MMM-'
'..MMM`..:::-...`MMM-.'
'..MMMMMMMMMMMMMM-'
  ``:::-``
```

```
pitatir@pitatir-Lenovo-Legion-Y9000P
-----
OS: Linux Mint 21.3 x86_64
Host: 82JD Lenovo Legion Y9000P2021H
Kernel: 5.15.0-100-generic
Uptime: 3 hours, 36 mins
Packages: 2507 (dpkg), 8 (flatpak)
Shell: bash 5.1.16
Resolution: 2928x1830
DE: Cinnamon 6.0.4
WM: Mutter (Muffin)
WM Theme: Mint-L-Dark-Purple (Mint-Y)
Theme: Mint-L-Dark-Purple [GTK2/3]
Icons: Mint-L-Purple [GTK2/3]
Terminal: gnome-terminal
CPU: 11th Gen Intel i7-11800H (16) @
GPU: Intel TigerLake-H GT1 [UHD Grap
GPU: NVIDIA GeForce RTX 3060 Mobile
Memory: 6176MiB / 31875MiB
```



Рис. 1 — Информация о системе

Обычно для динамического анализа используется инструментальное средство LTTng. К сожалению, при использовании утилиты возникли проблемы, которые не удалось устранить переустановкой библиотек.

Лог ошибок трассировщика ядра при использовании LTTng представлен ниже:

- Ошибка при попытке просмотреть список доступных для трассировки событий:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~$ sudo lttng list --kernel
Error: Unable to list kernel events: Kernel tracer not available
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~$
```

- Ошибка при попытке активировать события:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~$ sudo lttng create session
Session session created.
Traces will be output to /root/lttng-traces/session-20240529-000706
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~$ sudo lttng enable-event -k -a
Error: Events: Kernel tracer not available (channel channel0, session session)
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~$
```

В связи с невозможностью применения утилиты LTTng воспользуемся встроенным трассировщиком ядра ftrace.

1. Исследуем порождение процессов

Были написаны 2 простейшие программы:

- Программа родительского процесса, которая выводит информацию о текущем PID и PPID, порождает дочерний процесс и запускает новую программу на выполнение и ждет его завершения.
- Программа дочернего процесса выводит информацию о своем PID и PPID и засыпает на 2 секунды.

Файл 1-father.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
```

```

        // Вывод сообщения
        printf("Родительский процесс: pid = %d ppid = %d\n",
getpid(), getppid());

        // Если процесс дочерний
        if (fork() == 0)
            // Запустить дочерний процесс с аргументом
            execl("/home/pitativ/LR7/1s",
"/home/pitativ/LR7/1s", NULL);

        // Ожидание завершения дочернего процесса
        wait(NULL);
        // Вывод сообщения
        printf("Процесс pid = %d завершен\n", getpid());
        return 0;
}

```

Файл 1-son.c

```

#include <stdio.h>
#include <unistd.h>

int main() {
    // Вывод сообщения
    printf("Дочерний процесс: pid = %d ppid = %d\n",
getpid(), getppid());
    // Приостановка процесса на 2 секунды
    sleep(2);
    return 0;
}

```

Также был написан скрипт, который осуществляет трассировку функций ядра во время работы программ. В связи с тем, что полное содержимое файла с трассировкой при работе кода чересчур велико, ограничимся записью в трассировочный файл только тех системных функций, которые используются в описанных выше программах, а именно: `getpid()`, `getppid()`, `fork()`, `exec()`, `sleep()`, `wait()`. Стоит отметить, что функционал `ftrace` не предполагает записи в лог желаемых пользователем заданий, как это можно сделать с функциями с помощью `set_ftrace_filter`, поэтому фильтрацию полученного файла придется проводить отдельно с помощью утилиты `grep`.

Файл ftrace.sh

```
#!/bin/sh

# директория с необходимыми для трассировки файлами
dir=/sys/kernel/debug/tracing

# включаем трассировку
sysctl kernel.ftrace_enabled=1

# передаем в фильтр желаемые для отображения системные функции
echo '*getpid' > ${dir}/set_ftrace_filter
echo '*getppid' >> ${dir}/set_ftrace_filter
echo '*fork' >> ${dir}/set_ftrace_filter
echo '*exec' >> ${dir}/set_ftrace_filter
echo '*sleep' >> ${dir}/set_ftrace_filter
echo 'syscalls:.*wait' >> ${dir}/set_ftrace_filter

# выбираем трассировщик функций
echo function > ${dir}/current_tracer
# включаем обновление кольцевого буфера
echo 1 > ${dir}/tracing_on
# запускаем программу для трассировки
/home/pitatir/LR7/1f
# выключаем обновление кольцевого буфера
echo 0 > ${dir}/tracing_on

# выводим содержимое файла с трассировкой
less ${dir}/trace
```

Теперь проведем трассировку:

Перейдем в режим пользователя root

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:/$ sudo su
```

Перейдем в каталог с информацией о трассировке

```
root@pitatir-Lenovo-Legion-Y9000P2021H:/# cd /sys/kernel/debug/tracing
```

Запустим скрипт, осуществляющий трассировку

```
root@pitatir-Lenovo-Legion-Y9000P2021H:/sys/kernel/debug/tracing#
/home/pitatir/LR7/ftrace.sh
kernel.ftrace_enabled = 1
Родительский процесс: pid = 18244 ppid = 18241
Дочерний процесс: pid = 18245 ppid = 18244
Процесс pid = 18244 завершен
root@pitatir-Lenovo-Legion-Y9000P2021H:/sys/kernel/debug/tracing#
```

Исходный лог при вызове следующей утилиты отражен в файле “1-log.txt”.

В отчете оформим тот же вывод иначе для удобства чтения.

```
root@pitatir-Lenovo-Legion-Y9000P2021H:/sys/kernel/debug/tracing#
cat trace | grep -E '1f|1s'
```

Для начала ознакомимся с форматом вывода логов `ftrace`. Первые 13 строк общего лога выглядят так:

```
# tracer: function
#
# entries-in-buffer/entries-written: 61205/61205      #P:16
#
#      _-----> irqs-off
#     /_-----> need-resched
#    | /_-----> hardirq/softirq
#   || /_-----> preempt-depth
#  ||| /_-----> migrate-disable
# |||| /_-----> delay
# TASK-PID      CPU#  |||||  TIMESTAMP  FUNCTION
#| | | | | | | | | | | | | | | | | | | | | |
#      cinnamon-1660      [004] ..... 20636.781280:   pollwait <-unix pol
```

В первой строке указан используемый трассировщик. В данной ситуации это трассировщик функций `function`.

В следующей строке через слэш указано количество записей событий в буфере и общее количество записанных событий. Разница между этими двумя цифрами — это количество событий, утерянных при заполнении буфер, но в текущем случае утерянных событий нет.

Далее в трассировочной файле указывается текущее задание (TASK); его PID; номер процессорного ядра, на котором выполняется трассировка (CPU#); метка времени, показывающая, когда произошел вход в функцию (TIMESTAMP); имя трассируемой функции и имя родительской функции, которая ее вызвала (FUNCTION, здесь функцию `__pollwait` вызывает функция `unix_poll`). Символ `d` в поле `irqs-off` указывает на запрет прерываний. Остальные поля между CPU# и TIMESTAMP, обозначаемые точками по умолчанию, не потребуются при трассировке программ в учебных целях.

Теперь перейдем к анализу файла с трассировкой, полученного при исследовании порождения процессов. Еще раз отмечу, что в логе представлены только вызовы некоторых системных функций родительской и дочерней программами. Жирным шрифтом выделены строки с

получением PID и PPID для более простого определения трассируемого участка кода.

```
1f-18244 [010] ..... 20636.781503: path_noexec <-may_open
1f-18244 [010] ..... 20636.781509: sched_exec <-bprm_execve.part.0
1f-18244 [010] ..... 20636.781514: security_bprm_creds_for_exec <-
bprm_execve.part.0
1f-18244 [010] ..... 20636.781514: apparmor_bprm_creds_for_exec <-
security_bprm_creds_for_exec
1f-18244 [010] ..... 20636.781538: open_exec <-load_elf_binary
1f-18244 [010] ..... 20636.781545: path_noexec <-may_open
1f-18244 [010] ..... 20636.781551: begin_new_exec <-load_elf_binary
1f-18244 [010] ..... 20636.781561: do_close_on_exec <-begin_new_exec
1f-18244 [010] ..... 20636.781565: setup_new_exec <-load_elf_binary
1f-18244 [010] ..... 20636.781566: arch_setup_new_exec <-setup_new_exec
1f-18244 [010] ..... 20636.781593: path_noexec <-do_mmap
1f-18244 [010] ..... 20636.781604: path_noexec <-do_mmap
1f-18244 [010] ..... 20636.781605: path_noexec <-do_mmap
1f-18244 [010] ..... 20636.781608: path_noexec <-do_mmap
1f-18244 [010] ..... 20636.781622: path_noexec <-do_mmap
1f-18244 [010] ..... 20636.781634: path_noexec <-do_mmap
1f-18244 [010] ..... 20636.781636: path_noexec <-do_mmap
1f-18244 [010] ..... 20636.781639: path_noexec <-do_mmap
1f-18244 [010] ..... 20636.781663: finalize_exec <-load_elf_binary
1f-18244 [010] ..... 20636.781797: path_noexec <-do_mmap
1f-18244 [010] ..... 20636.781830: path_noexec <-do_mmap
1f-18244 [010] ..... 20636.781851: path_noexec <-do_mmap
1f-18244 [010] ..... 20636.781862: path_noexec <-do_mmap
1f-18244 [010] ..... 20636.781866: path_noexec <-do_mmap
1f-18244 [010] ..... 20636.782116: __do_sys_getppid <-x64_sys_call
1f-18244 [010] ..... 20636.782117: __do_sys_getpid <-x64_sys_call
1f-18244 [010] ..... 20636.782169: tty_ldisc_ref_wait <-
file_tty_write.constprop.0
1f-18244 [010] ..... 20636.782194: cgroup_fork <-copy_process
1f-18244 [010] ..... 20636.782194: sched_fork <-copy_process
1f-18244 [010] ..... 20636.782194: __sched_fork <-sched_fork
1f-18244 [010] ..... 20636.782201: tty_audit_fork <-copy_signal
1f-18244 [010] ..... 20636.782201: sched_autogroup_fork <-copy_signal
1f-18244 [010] ..... 20636.782206: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782207: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782208: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782208: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782216: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782218: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782222: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782227: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782228: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782229: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782230: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782230: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782236: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782240: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782242: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782244: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782245: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782245: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782247: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782249: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782250: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782254: anon_vma_fork <-dup_mmap
```

```

1f-18244 [010] ..... 20636.782256: anon_vma_fork <-dup_mmap
1f-18244 [010] ..... 20636.782261: cgroup_can_fork <-copy_process
1f-18244 [010] ..... 20636.782261: cgroup_css_set_fork <-cgroup_can_fork
1f-18244 [010] ..... 20636.782261: pids_can_fork <-cgroup_can_fork
1f-18244 [010] ..... 20636.782263: sched_cgroup_fork <-copy_process
1f-18244 [010] d.... 20636.782265: sched_core_fork <-copy_process
1f-18244 [010] ..... 20636.782266: sched_post_fork <-copy_process
1f-18244 [010] ..... 20636.782266: cgroup_post_fork <-copy_process
1f-18244 [010] ..... 20636.782267: cpuset_fork <-cgroup_post_fork
1f-18244 [010] ..... 20636.782267: cpu_cgroup_fork <-cgroup_post_fork
1f-18244 [010] ..... 20636.782272: freezer_fork <-cgroup_post_fork
1f-18244 [010] ..... 20636.782272: cgroup_css_set_put_fork <-
cgroup_post_fork
1f-18244 [010] ..... 20636.782296: do_wait <-kernel_wait4
1s-18245 [011] ..... 20636.782508: path_noexec <-may_open
1s-18245 [011] ..... 20636.782514: sched_exec <-bprm_execve.part.0
1s-18245 [011] ..... 20636.782520: security_bprm_creds_for_exec <-
bprm_execve.part.0
1s-18245 [011] ..... 20636.782520: apparmor_bprm_creds_for_exec <-
security_bprm_creds_for_exec
1s-18245 [011] ..... 20636.782538: open_exec <-load_elf_binary
1s-18245 [011] ..... 20636.782544: path_noexec <-may_open
1s-18245 [011] ..... 20636.782547: begin_new_exec <-load_elf_binary
1s-18245 [011] ..... 20636.782610: do_close_on_exec <-begin_new_exec
1s-18245 [011] ..... 20636.782613: setup_new_exec <-load_elf_binary
1s-18245 [011] ..... 20636.782615: arch_setup_new_exec <-setup_new_exec
1s-18245 [011] ..... 20636.782637: path_noexec <-do_mmap
1s-18245 [011] ..... 20636.782644: path_noexec <-do_mmap
1s-18245 [011] ..... 20636.782646: path_noexec <-do_mmap
1s-18245 [011] ..... 20636.782647: path_noexec <-do_mmap
1s-18245 [011] ..... 20636.782658: path_noexec <-do_mmap
1s-18245 [011] ..... 20636.782666: path_noexec <-do_mmap
1s-18245 [011] ..... 20636.782668: path_noexec <-do_mmap
1s-18245 [011] ..... 20636.782673: path_noexec <-do_mmap
1s-18245 [011] ..... 20636.782700: finalize_exec <-load_elf_binary
1s-18245 [011] ..... 20636.782831: path_noexec <-do_mmap
1s-18245 [011] ..... 20636.782859: path_noexec <-do_mmap
1s-18245 [011] ..... 20636.782870: path_noexec <-do_mmap
1s-18245 [011] ..... 20636.782878: path_noexec <-do_mmap
1s-18245 [011] ..... 20636.782883: path_noexec <-do_mmap
1s-18245 [011] ..... 20636.783115: __do_sys_getppid <-x64_sys_call
1s-18245 [011] ..... 20636.783116: __do_sys_getpid <-x64_sys_call
1s-18245 [011] ..... 20636.783170: tty_ldisc_ref_wait <-
file_tty_write.constprop.0
1s-18245 [011] ..... 20636.783179: __x64_sys_clock_nanosleep <-
x64_sys_call
1s-18245 [011] ..... 20636.783180: common_nsleep <-
__x64_sys_clock_nanosleep
1s-18245 [011] ..... 20636.783180: hrtimer_nanosleep <-common_nsleep
1s-18245 [011] ..... 20636.783181: do_nanosleep <-hrtimer_nanosleep
1f-18244 [010] ..... 20638.783625: __do_sys_getpid <-x64_sys_call
1f-18244 [010] ..... 20638.783649: tty_ldisc_ref_wait <-
file_tty_write.constprop.0
root@pitatir-Lenovo-Legion-Y9000P2021H:/sys/kernel/debug/tracing#

```

Начнем с того, что родительский и дочерний процесс выполнялись последовательно: сначала 1f с PID 18244, потом 1s с PID 18245, и снова процесс 1f, которому нужно завершить свою работу. PID и PPID процессов

в выводе программы в терминал и в трассировке совпадают, как и должно быть.

Родительский процесс все время выполнялся на 10м ядре, а дочерний на 11ом. Можно предположить, это проявление мультипроцессорной обработки, когда работа системы организована так, что несколько процессов выполняются одновременно и параллельно на разных обрабатывающих устройствах системы (в данной ситуации, на разных ядрах).

При создании дочернего процесса `ls` видно, что сначала был создан дочерний процесс с PID 18245, и во время его работы (на втором этапе создания) с помощью функции `exec()` был заполнен контекст.

Проанализируем функции трассировки, указанные в столбце FUNCTION:

- `path_noexec`: ядро проверяет, должен ли открываемый файл обрабатываться как "не выполняемый", т.е. может ли он быть выполнен как код. Это часть мер безопасности, предотвращающих выполнение вредоносного кода.
- `sched_exec`: планирование выполнения процесса, подготовка к переключению контекста.
- `security_bprm_creds_for_exec`: настройка контекста безопасности
- `apparmor_bprm_creds_for_exec`: подготовка учетных данных безопасности для модуля безопасности ядра Linux.
- `open_exec`: открытие исполняемого файла для запуска: поиск файла на диске, проверка ограничений, подготовка файла к исполнению.
- `begin_new_exec`: переход от старого процесса к новому
- `do_close_on_exec`: закрывает файловые дескрипторы в новом образе процесса, изолируя новый процесс от старого.
- `setup_new_exec`: настройка нового образа процесса для выполнения: копирование кода программы и данных в память, настройка стека.

- `arch_setup_new_exec`: настройка нового исполняемого файла для конкретной архитектуры.
- `finalize_exec`: завершение выполнения новой программы, очистка оставшихся ресурсов и пометка процесса как завершённый.
- `__do_sys_getppid` и `__do_sys_getpid`: получение идентификатора родительского процесса и текущего идентификатора процесса соответственно.
- `tty_ldisc_ref_wait`: ожидание ссылки на линию терминала, которая является частью обработки операций ввода/вывода.
- `cgroup_fork`: связывание нового процесса с cgroup его родительского процесса. Определяет, как ресурсы распределяются и управляются для нового процесса в контексте системы cgroup.
- `sched_fork`: обрабатывает аспекты планирования для вновь созданного процесса, отвечает за наследование параметров планирования от родительского процесса.
- `cgroup_post_fork`: завершает объединение cgroup для нового процесса.
- `sched_post_fork`: завершает настройку планирования нового процесса.
- `cpuset_fork`: гарантирует, что новый процесс наследует настройки привязки к процессору своего родительского процесса.
- `cpu_ные` с переходом в спящий режим (приостановкой выполнения).
- `cgroup_fork`: наследование политики распределения ЦП и планирования от своей контрольной группы.
- `freezer_fork`: заморозка и разморозка процесса.
- `cgroup_css_set_fork`, `cgroup_css_set_put_fork`: обработкой подсистемой cgroup общих ресурсов между процессами.
- `anon_vma_fork`: обработка анонимных отображений памяти для дочернего процесса во время fork.
- `__x64_sys_clock_nanosleep`, `common_nsleep`, `hrtimer_nanosleep`, `do_nanosleep`: функции, связан

2. Исследуем переключение процессов (смену контекста)

Воспользуемся программами из пункта 1, но поменяем скрипт трассировки ядра так, чтобы в нем отображалась смена контекста.

Файл ftrace2.sh

```
#!/bin/sh

# директория с необходимыми для трассировки файлами
dir=/sys/kernel/debug/tracing

# включаем трассировку
sysctl kernel.ftrace_enabled=1

# передаем в фильтр желаемые для отображения системные функции
echo '*getpid' > ${dir}/set_ftrace_filter
echo '*getppid' >> ${dir}/set_ftrace_filter
echo '*fork' >> ${dir}/set_ftrace_filter
echo '*exec' >> ${dir}/set_ftrace_filter
echo '*sleep' >> ${dir}/set_ftrace_filter
echo '*wait' >> ${dir}/set_ftrace_filter
echo '*waitid' >> ${dir}/set_ftrace_filter
echo '*waitpid' >> ${dir}/set_ftrace_filter

# выбираем трассировщик функций
echo function > ${dir}/current_tracer

# включаем анализ событий переключения планировщика
# т.е. переключений контекста
echo 1 > events/sched/sched_switch/enable
# включаем анализ событий пробуждения планировщика
# т.е. запуска процесса после перехода в режим ожидания или блокировки
echo 1 > events/sched/sched_wakeup/enable

# включаем обновление кольцевого буфера
echo 1 > ${dir}/tracing_on
# запускаем программу для трассировки
/home/pitativ/LR7/1f
# выключаем обновление кольцевого буфера
echo 0 > ${dir}/tracing_on

# выводим содержимое файла с трассировкой
less ${dir}/trace
```

Запустим скрипт, проанализируем содержимое файла с трассировкой.

Строки с новыми трассируемыми функциями: sched_switch, которая

отвечает за переключение контекста, и `sched_wakeur`, которая отвечает за запуск процесса после перехода в режим ожидания или блокировки, выделены жирным шрифтом и курсивом соответственно.

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:/$ sudo su
root@pitatir-Lenovo-Legion-Y9000P2021H:/# cd /sys/kernel/debug/tracing
root@pitatir-Lenovo-Legion-Y9000P2021H:/sys/kernel/debug/tracing#
/home/pitatir/LR7/ftrace2.sh
kernel.ftrace_enabled = 1
Родительский процесс: pid = 25570 ppid = 25568
Дочерний процесс: pid = 25571 ppid = 25570
Процесс pid = 25570 завершен
root@pitatir-Lenovo-Legion-Y9000P2021H:/sys/kernel/debug/tracing# cat trace
| grep -E '1f|ls'
1f-25570    [006]    76834506660707: path_noexec <-may_open
1f-25570    [006]    76834506677971: sched_exec <-bprm_execve.part.0
1f-25570    [006]    76834506688769:  security_bprm_creds_for_exec  <-
bprm_execve.part.0
1f-25570    [006]    76834506689809:  apparmor_bprm_creds_for_exec  <-
security_bprm_creds_for_exec
1f-25570    [006]    76834506751797: open_exec <-load_elf_binary
1f-25570    [006]    76834506770101: path_noexec <-may_open
1f-25570    [006]    76834506782833: begin_new_exec <-load_elf_binary
1f-25570    [006]    76834506811005: do_close_on_exec <-begin_new_exec
1f-25570    [006]    76834506820229: setup_new_exec <-load_elf_binary
1f-25570    [006]    76834506826997: arch_setup_new_exec <-setup_new_exec
1f-25570    [006]    76834506906597: path_noexec <-do_mmap
1f-25570    [006]    76834506929079: path_noexec <-do_mmap
1f-25570    [006]    76834506933579: path_noexec <-do_mmap
1f-25570    [006]    76834506939033: path_noexec <-do_mmap
1f-25570    [006]    76834506996197: path_noexec <-do_mmap
1f-25570    [006]    76834507029433: path_noexec <-do_mmap
1f-25570    [006]    76834507033325: path_noexec <-do_mmap
1f-25570    [006]    76834507040779: path_noexec <-do_mmap
1f-25570    [006]    76834507100113: finalize_exec <-load_elf_binary
1f-25570    [006]    76834507451609: path_noexec <-do_mmap
1f-25570    [006]    76834507541783: path_noexec <-do_mmap
1f-25570    [006]    76834507587689: path_noexec <-do_mmap
1f-25570    [006]    76834507614601: path_noexec <-do_mmap
1f-25570    [006]    76834507625963: path_noexec <-do_mmap
1f-25570    [006]    76834508287733: __do_sys_getppid <-x64_sys_call
1f-25570    [006]    76834508290459: __do_sys_getpid <-x64_sys_call
1f-25570    [006]    76834508414837:  tty_ldisc_ref_wait  <-
file_tty_write.constprop.0
1f-25570    [006]    76834508512175: cgroup_fork <-copy_process
1f-25570    [006]    76834508512837: sched_fork <-copy_process
1f-25570    [006]    76834508513409:  __sched_fork <-sched_fork
1f-25570    [006]    76834508554065:  tty_audit_fork <-copy_signal
1f-25570    [006]    76834508554647: sched_autogroup_fork <-copy_signal
1f-25570    [006]    76834508564675: anon_vma_fork <-dup_mmap
1f-25570    [006]    76834508567681: anon_vma_fork <-dup_mmap
1f-25570    [006]    76834508569593: anon_vma_fork <-dup_mmap
1f-25570    [006]    76834508571645: anon_vma_fork <-dup_mmap
1f-25570    [006]    76834508587437: anon_vma_fork <-dup_mmap
1f-25570    [006]    76834508592229: anon_vma_fork <-dup_mmap
1f-25570    [006]    76834508600285: anon_vma_fork <-dup_mmap
1f-25570    [006]    76834508610961: anon_vma_fork <-dup_mmap
```

```

1f-25570 [006] 76834508613017: anon_vma_fork <-dup_mmap
1f-25570 [006] 76834508614999: anon_vma_fork <-dup_mmap
1f-25570 [006] 76834508616605: anon_vma_fork <-dup_mmap
1f-25570 [006] 76834508618315: anon_vma_fork <-dup_mmap
1f-25570 [006] 76834508625789: anon_vma_fork <-dup_mmap
1f-25570 [006] 76834508630357: anon_vma_fork <-dup_mmap
1f-25570 [006] 76834508637923: anon_vma_fork <-dup_mmap
1f-25570 [006] 76834508642267: anon_vma_fork <-dup_mmap
1f-25570 [006] 76834508644999: anon_vma_fork <-dup_mmap
1f-25570 [006] 76834508647239: anon_vma_fork <-dup_mmap
1f-25570 [006] 76834508651187: anon_vma_fork <-dup_mmap
1f-25570 [006] 76834508655841: anon_vma_fork <-dup_mmap
1f-25570 [006] 76834508661905: anon_vma_fork <-dup_mmap
1f-25570 [006] 76834508671899: anon_vma_fork <-dup_mmap
1f-25570 [006] 76834508675307: anon_vma_fork <-dup_mmap
1f-25570 [006] 76834508688181: cgroup_can_fork <-copy_process
1f-25570 [006] 76834508688755: cgroup_css_set_fork <-cgroup_can_fork
1f-25570 [006] 76834508690059: pids_can_fork <-cgroup_can_fork
1f-25570 [006] 76834508701501: sched_cgroup_fork <-copy_process
1f-25570 [006] 76834508707445: sched_core_fork <-copy_process
1f-25570 [006] 76834508710201: sched_post_fork <-copy_process
1f-25570 [006] 76834508710707: cgroup_post_fork <-copy_process
1f-25570 [006] 76834508712781: cpuset_fork <-cgroup_post_fork
1f-25570 [006] 76834508713955: cpu_cgroup_fork <-cgroup_post_fork
1f-25570 [006] 76834508715881: freezer_fork <-cgroup_post_fork
1f-25570 [006] 76834508716501: cgroup_css_set_put_fork <-cgroup_post_fork
1f-25570 [006] 76834508778009: do_wait <-kernel_wait4
1f-25570 [006] 76834508800249: sched_switch: prev_comm=1f prev_pid=25570
prev_prio=120 prev_state=S ==> next_comm=swapper/6 next_pid=0 next_prio=120
<idle>-0 [013] 76834508937525: sched_switch: prev_comm=swapper/13
prev_pid=0 prev_prio=120 prev_state=R ==> next_comm=1f next_pid=25571
next_prio=120
1s-25571 [013] 76834509253741: path_noexec <-may_open
1s-25571 [013] 76834509269777: sched_exec <-bprm_execve.part.0
1s-25571 [013] 76834509283405: security_bprm_creds_for_exec <-
bprm_execve.part.0
1s-25571 [013] 76834509284253: apparmor_bprm_creds_for_exec <-
security_bprm_creds_for_exec
1s-25571 [013] 76834509327873: open_exec <-load_elf_binary
1s-25571 [013] 76834509341927: path_noexec <-may_open
1s-25571 [013] 76834509352159: begin_new_exec <-load_elf_binary
1s-25571 [013] 76834509514049: do_close_on_exec <-begin_new_exec
1s-25571 [013] 76834509522069: setup_new_exec <-load_elf_binary
1s-25571 [013] 76834509527821: arch_setup_new_exec <-setup_new_exec
1s-25571 [013] 76834509568441: path_noexec <-do_mmap
1s-25571 [013] 76834509589119: path_noexec <-do_mmap
1s-25571 [013] 76834509593337: path_noexec <-do_mmap
1s-25571 [013] 76834509597009: path_noexec <-do_mmap
1s-25571 [013] 76834509632843: path_noexec <-do_mmap
1s-25571 [013] 76834509664029: path_noexec <-do_mmap
1s-25571 [013] 76834509667645: path_noexec <-do_mmap
1s-25571 [013] 76834509674139: path_noexec <-do_mmap
1s-25571 [013] 76834509730047: finalize_exec <-load_elf_binary
1s-25571 [013] 76834510045553: path_noexec <-do_mmap
1s-25571 [013] 76834510119941: path_noexec <-do_mmap
1s-25571 [013] 76834510156525: path_noexec <-do_mmap
1s-25571 [013] 76834510178701: path_noexec <-do_mmap
1s-25571 [013] 76834510189745: path_noexec <-do_mmap
1s-25571 [013] 76834510785107: __do_sys_getppid <-x64_sys_call
1s-25571 [013] 76834510787747: __do_sys_getpid <-x64_sys_call

```

```

1s-25571          [013]          76834511006795:      tty_ldisc_ref_wait      <-
file_tty_write.constprop.0
1s-25571          [013]  76834511054229:  __x64_sys_clock_nanosleep <-__x64_sys_call
1s-25571          [013]  76834511056175:  common_nsleep <-__x64_sys_clock_nanosleep
1s-25571          [013]  76834511056649:  hrtimer_nanosleep <-common_nsleep
1s-25571          [013]  76834511057501:  do_nanosleep <-hrtimer_nanosleep
1s-25571          [013]  76834511074217:  sched_switch: prev_comm=1s prev_pid=25571
prev_prio=120 prev_state=S ==> next_comm=swapper/13 next_pid=0 next_prio=120
<idle>-0          [013]  76839119409693:  sched_wakeup: comm=1s pid=25571 prio=120
target_cpu=013
<idle>-0          [013]  76839119442193:  sched_switch: prev_comm=swapper/13
prev_pid=0 prev_prio=120 prev_state=R ==> next_comm=1s next_pid=25571
next_prio=120
1s-25571          [013]  76839119938621:  sched_switch: prev_comm=1s prev_pid=25571
prev_prio=120 prev_state=Z ==> next_comm=swapper/13 next_pid=0 next_prio=120
<idle>-0          [006]  76839120073333:  sched_wakeup: comm=1f pid=25570 prio=120
target_cpu=006
<idle>-0          [006]  76839120087673:  sched_switch: prev_comm=swapper/6
prev_pid=0 prev_prio=120 prev_state=R ==> next_comm=1f next_pid=25570
next_prio=120
1f-25570          [006]  76839120174283:  __do_sys_getpid <-__x64_sys_call
1f-25570          [006]  76839120265741:  tty_ldisc_ref_wait      <-
file_tty_write.constprop.0
1f-25570          [006]  76839120689461:  sched_switch: prev_comm=1f prev_pid=25570
prev_prio=120 prev_state=Z ==> next_comm=swapper/6 next_pid=0 next_prio=120
root@pitatir-Lenovo-Legion-Y9000P2021H:/sys/kernel/debug/tracing#

```

Проанализируем содержимое строки из лога с изменением контекста с помощью sched_switch: prev_comm - предыдущая команда, prev_pid - предыдущий pid, prev_prio - предыдущий приоритет, prev_state - предыдущее состояние, next_comm - следующая команда, next_pid - следующее состояние, next_prio - предыдущий приоритет.

Переключение контекста вызывается при переходе к дочернему процессу: сначала из спящего процесса 1f-25570 вызывается swapper/6, который является частью ядра (next_pid=0) и обрабатывает замену страниц памяти на диск, а затем происходит переход к дочернему процессу с pid=25571. Дочерний процесс наследует приоритетность родительского.

```

1f-25570          [006]  76834508800249:  sched_switch: prev_comm=1f prev_pid=25570
prev_prio=120 prev_state=S ==> next_comm=swapper/6 next_pid=0 next_prio=120
<idle>-0          [013]  76834508937525:  sched_switch: prev_comm=swapper/13
prev_pid=0 prev_prio=120 prev_state=R ==> next_comm=1f next_pid=25571
next_prio=120

```

Переключение контекста также происходит при засыпании и пробуждении дочернего процесса с помощью sleep():

```

1s-25571 [013] 76834511074217: sched_switch: prev_comm=1s prev_pid=25571
prev_prio=120 prev_state=S ==> next_comm=swapper/13 next_pid=0 next_prio=120
<idle>-0 [013] 76839119409693: sched_wakeup: comm=1s pid=25571 prio=120
target_cpu=013
<idle>-0 [013] 76839119442193: sched_switch: prev_comm=swapper/13
prev_pid=0 prev_prio=120 prev_state=R ==> next_comm=1s next_pid=25571
next_prio=120

```

Из лога видно, как сначала спящий дочерний процесс (prev_state=S) был разбужен ядром с помощью (sched_wakeup) и переведен в режим выполнения (prev_state=R).

Далее контекст переключается для возвращения к родительскому процессу от дочернего, родительский процесс при этом так же пробуждается при помощи sched_wakeup:

```

1s-25571 [013] 76839119938621: sched_switch: prev_comm=1s prev_pid=25571
prev_prio=120 prev_state=Z ==> next_comm=swapper/13 next_pid=0 next_prio=120
<idle>-0 [006] 76839120073333: sched_wakeup: comm=1f pid=25570 prio=120
target_cpu=006
<idle>-0 [006] 76839120087673: sched_switch: prev_comm=swapper/6
prev_pid=0 prev_prio=120 prev_state=R ==> next_comm=1f next_pid=25570
next_prio=120

```

В последний раз смена контекста вызывается при завершении родительского процесса:

```

1f-25570 [006] 76839120689461: sched_switch: prev_comm=1f prev_pid=25570
prev_prio=120 prev_state=Z ==> next_comm=swapper/6 next_pid=0 next_prio=120

```

Если в файле /sys/kernel/tracing/trace дальше посмотреть строки лога, то можно увидеть, как пробуждается процесс со скриптом трассирования, а затем терминал и контекст сменяется на процесс терминала:

```

kworker/u32:1-24868 [010] 79153045284951: sched_wakeup: comm=ftrace2.sh
pid=26037 prio=120 target_cpu=010
kworker/u32:1-24868 [010] 79153045313077: sched_switch:
prev_comm=kworker/u32:1 prev_pid=24868 prev_prio=120 prev_state=I ==>
next_comm=ftrace2.sh next_pid=26037 next_prio=120
<idle>-0 [000] 79153045319151: sched_wakeup: comm=gnome-terminal-
pid=5494 prio=120 target_cpu=000
<idle>-0 [000] 79153045324101: sched_switch: prev_comm=swapper/0
prev_pid=0 prev_prio=120 prev_state=R ==> next_comm=gnome-terminal-
next_pid=5494 next_prio=120

```

3. Исследуем конкуренцию процессов при разных стратегиях планирования и разном количестве ядер.

Была написана программа, создающая 32 дочерних процесса, проводящая в каждом из них счет до 300000000 с информированием о запуске процесса, достижения им середины подсчетов и завершении работы. Родительский процесс дожидается завершения каждого дочернего процесса. Программой предусмотрена возможность смены политики планирования.

Файл 3.c

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sched.h>
#include <sys/mman.h>
#include <sys/wait.h>

int main(void) {
    // Параметры планирования
    struct sched_param shed_parameters;
    // Pid завершеного процесса
    pid_t status;
    // Статус изменения планировщика
    int res;

    // Получаем параметры планирования текущего
    // родительского процесса
    sched_getparam(0, &shed_parameters);

    // Установка приоритета планирования
    shed_parameters.sched_priority = 50;

    // Меняем политику планирования
    res = sched_setscheduler(0, SCHED_FIFO,
    &shed_parameters);
    // Если попытка оказалась неудачной
    if (res == -1) {
        // Вывод сообщения об ошибке
        perror("SCHED_SETSCHEDULER_1");
    }

    // Выводим измененную политику планирования
    printf("Политика планирования родительского процесса:");
    switch (sched_getscheduler(0)) {
        case SCHED_FIFO:
            printf("SCHED_FIFO\n\n");
            break;
    }
}
```



```

        case SCHED_RR:
            printf("SCHED_RR\n\n");
            break;
        case SCHED_OTHER:
            printf("SCHED_OTHER\n\n");
            break;
        case -1:
            perror("SCHED_GETSCHEDULER\n");
            break;
        default:
            printf("Неизвестная политика
планирования\n\n");
    }

    // Создаем 32 дочерних процесса
    for (int i = 0; i < 32; i++)
        // Если процесс дочерний
        if (fork() == 0) {
            // Выводим сообщение
            printf("%d) Дочерний процесс: pid = %d ppid =
%d\n", i, getpid(), getppid());

            // Выполняем длинный цикл
            for (long j = 0; j < 3000000000; j++) {
                // Выводим сообщение о достижении середины
                if (j == 1500000000) {
                    printf("Checkpoint %d: процесс с pid =
%d\n", i, getpid());
                }
            }
            // Выводим сообщение о завершении работы
            // дочернего процесса
            printf("Процесс %i с pid = %i закончен\n", i,
getpid());
            // Завершаем работу
            exit(0);
        }

    // Ждем выполнения всех дочерних процессов
    while ((status = wait(NULL)) > 0) {
        printf("Процесс завершен: pid = %d\n", status);
    }
    // Выводим сообщение о завершении работы родительского
    // процесса
    printf("Завершение родительской программы pid = %d\n",
getpid());
    return 0;
}

```

Также был изменен скрипт, осуществляющий трассировку:

Файл ftrace3.sh

```
#!/bin/sh

# директория с необходимыми для трассировки файлами
dir=/sys/kernel/debug/tracing

# включаем трассировку
sysctl kernel.ftrace_enabled=1

# передаем в фильтр желаемые для отображения системные функции
echo '*getppid' > ${dir}/set_ftrace_filter
echo '*wait' >> ${dir}/set_ftrace_filter

# включаем опцию отображения PID процесса и задачи
echo funcgraph-proc > ${dir}/trace_options

# выбираем графический трассировщик функций
echo function_graph > ${dir}/current_tracer

# включаем анализ событий переключения планировщика
# т.е. переключений контекста
echo 1 > events/sched/sched_switch/enable
# включаем анализ событий пробуждения планировщика
# т.е. запуска процесса после перехода в режим ожидания или блокировки
echo 1 > events/sched/sched_wakeup/enable

# включаем обновление кольцевого буфера
echo 1 > ${dir}/tracing_on
# запускаем программу для трассировки
/home/pitatir/LR7/prog3
# выключаем обновление кольцевого буфера
echo 0 > ${dir}/tracing_on

# выводим содержимое файла с трассировкой
less ${dir}/trace
```

Здесь используется другой трассировщик `function_graph`, поэтому содержимое итогового файла с трассировкой отличается:

```
# tracer: function_graph
#
# CPU   TASK/PID          DURATION          FUNCTION CALLS
# |     |             |             |             |
1)      <idle>-0      |             |             /* sched_wakeup:
comm=Chrome_IOThread pid=3447 prio=120 target_cpu=001 */
```

Столбцы имеют следующие значения: номер процессора, задание и PID, продолжительность выполнения процесса, функция и ее родительская функция.

Запустим программу с планировщиком SHED_FIFO без ограничения количества используемых ядер:

```
root@pitatir-Lenovo-Legion-Y9000P2021H:/sys/kernel/debug/tracing#  
/home/pitatir/LR7/ftrace3.sh
```

```
kernel.ftrace_enabled = 1
```

```
Политика планирования родительского процесса: SCHED_FIFO
```

```
0) Дочерний процесс: pid = 30195 ppid = 30194  
2) Дочерний процесс: pid = 30197 ppid = 30194  
1) Дочерний процесс: pid = 30196 ppid = 30194  
3) Дочерний процесс: pid = 30198 ppid = 30194  
6) Дочерний процесс: pid = 30201 ppid = 30194  
4) Дочерний процесс: pid = 30199 ppid = 30194  
5) Дочерний процесс: pid = 30200 ppid = 30194  
7) Дочерний процесс: pid = 30202 ppid = 30194  
8) Дочерний процесс: pid = 30203 ppid = 30194  
9) Дочерний процесс: pid = 30204 ppid = 30194  
10) Дочерний процесс: pid = 30205 ppid = 30194  
11) Дочерний процесс: pid = 30206 ppid = 30194  
12) Дочерний процесс: pid = 30207 ppid = 30194  
13) Дочерний процесс: pid = 30208 ppid = 30194  
14) Дочерний процесс: pid = 30209 ppid = 30194  
15) Дочерний процесс: pid = 30210 ppid = 30194  
Checkpoint 5: процесс с pid = 30200  
Checkpoint 14: процесс с pid = 30209  
Checkpoint 2: процесс с pid = 30197  
Checkpoint 11: процесс с pid = 30206  
Checkpoint 8: процесс с pid = 30203  
Checkpoint 0: процесс с pid = 30195  
Checkpoint 3: процесс с pid = 30198  
Checkpoint 13: процесс с pid = 30208  
Checkpoint 6: процесс с pid = 30201  
Checkpoint 1: процесс с pid = 30196  
Checkpoint 10: процесс с pid = 30205  
Checkpoint 9: процесс с pid = 30204  
Checkpoint 4: процесс с pid = 30199  
Checkpoint 15: процесс с pid = 30210  
Checkpoint 7: процесс с pid = 30202  
Checkpoint 12: процесс с pid = 30207  
Процесс 5 с pid = 30200 закончен  
16) Дочерний процесс: pid = 30211 ppid = 30194  
Процесс 6 с pid = 30201 закончен  
17) Дочерний процесс: pid = 30212 ppid = 30194  
Процесс 14 с pid = 30209 закончен  
Процесс 1 с pid = 30196 закончен  
Процесс 2 с pid = 30197 закончен  
18) Дочерний процесс: pid = 30213 ppid = 30194  
Процесс 11 с pid = 30206 закончен  
19) Дочерний процесс: pid = 30214 ppid = 30194  
Процесс 0 с pid = 30195 закончен  
20) Дочерний процесс: pid = 30215 ppid = 30194
```

21) Дочерний процесс: pid = 30216 ppid = 30194
Процесс 3 с pid = 30198 закончен
22) Дочерний процесс: pid = 30217 ppid = 30194
23) Дочерний процесс: pid = 30218 ppid = 30194
Процесс 8 с pid = 30203 закончен
24) Дочерний процесс: pid = 30219 ppid = 30194
Процесс 7 с pid = 30202 закончен
Процесс 10 с pid = 30205 закончен
Процесс 9 с pid = 30204 закончен
25) Дочерний процесс: pid = 30220 ppid = 30194
26) Дочерний процесс: pid = 30221 ppid = 30194
27) Дочерний процесс: pid = 30222 ppid = 30194
Процесс 4 с pid = 30199 закончен
28) Дочерний процесс: pid = 30223 ppid = 30194
Процесс 13 с pid = 30208 закончен
29) Дочерний процесс: pid = 30224 ppid = 30194
Процесс 15 с pid = 30210 закончен
30) Дочерний процесс: pid = 30225 ppid = 30194
Процесс 12 с pid = 30207 закончен
31) Дочерний процесс: pid = 30226 ppid = 30194
Checkpoint 16: процесс с pid = 30211
Checkpoint 18: процесс с pid = 30213
Checkpoint 22: процесс с pid = 30217
Checkpoint 17: процесс с pid = 30212
Checkpoint 23: процесс с pid = 30218
Checkpoint 19: процесс с pid = 30214
Checkpoint 20: процесс с pid = 30215
Checkpoint 21: процесс с pid = 30216
Checkpoint 24: процесс с pid = 30219
Checkpoint 27: процесс с pid = 30222
Checkpoint 28: процесс с pid = 30223
Checkpoint 29: процесс с pid = 30224
Checkpoint 26: процесс с pid = 30221
Checkpoint 25: процесс с pid = 30220
Checkpoint 30: процесс с pid = 30225
Checkpoint 31: процесс с pid = 30226
Процесс 16 с pid = 30211 закончен
Процесс завершен: pid = 30195
Процесс завершен: pid = 30196
Процесс завершен: pid = 30197
Процесс завершен: pid = 30198
Процесс завершен: pid = 30199
Процесс завершен: pid = 30200
Процесс завершен: pid = 30201
Процесс завершен: pid = 30202
Процесс завершен: pid = 30203
Процесс завершен: pid = 30204
Процесс завершен: pid = 30205
Процесс завершен: pid = 30206
Процесс завершен: pid = 30207
Процесс завершен: pid = 30208
Процесс завершен: pid = 30209
Процесс завершен: pid = 30210
Процесс завершен: pid = 30211
Процесс 22 с pid = 30217 закончен
Процесс завершен: pid = 30217
Процесс 18 с pid = 30213 закончен
Процесс завершен: pid = 30213
Процесс 21 с pid = 30216 закончен
Процесс завершен: pid = 30216
Процесс 19 с pid = 30214 закончен

```

Процесс завершен: pid = 30214
Процесс 23 с pid = 30218 закончен
Процесс 20 с pid = 30215 закончен
Процесс завершен: pid = 30218
Процесс завершен: pid = 30215
Процесс 17 с pid = 30212 закончен
Процесс завершен: pid = 30212
Процесс 29 с pid = 30224 закончен
Процесс 24 с pid = 30219 закончен
Процесс завершен: pid = 30224
Процесс завершен: pid = 30219
Процесс 26 с pid = 30221 закончен
Процесс 27 с pid = 30222 закончен
Процесс завершен: pid = 30221
Процесс завершен: pid = 30222
Процесс 25 с pid = 30220 закончен
Процесс завершен: pid = 30220
Процесс 28 с pid = 30223 закончен
Процесс завершен: pid = 30223
Процесс 30 с pid = 30225 закончен
Процесс завершен: pid = 30225
Процесс 31 с pid = 30226 закончен
Процесс завершен: pid = 30226
Завершение родительской программы pid = 30194

```

Лог вывода трассировки чересчур велик. Приведем его конец для анализа:

```

0) prog3-30221 | 0.402 us | tty_ldisc_ref_wait();
15) prog3-30222 | 0.604 us | tty_ldisc_ref_wait();
10) <idle>-0 | /* sched_wakeup: comm=prog3 pid=30194
prio=49 target_cpu=010 */
0) prog3-30221 | /* sched_switch: prev_comm=prog3
prev_pid=30221 prev_prio=49 prev_state=Z ==> next_comm=kworker/0:1
next_pid=17 next_prio=120 */
10) <idle>-0 | /* sched_switch: prev_comm=swapper/10
prev_pid=0 prev_prio=120 prev_state=R ==> next_comm=prog3 next_pid=30194
next_prio=49 */
10) <idle>-0 => prog3-30194
10) prog3-30194 | 418.614 us | } /* do_wait */
10) prog3-30194 | 0.524 us | tty_ldisc_ref_wait();
15) prog3-30222 | /* sched_switch: prev_comm=prog3
prev_pid=30222 prev_prio=49 prev_state=Z ==> next_comm=kworker/15:1
next_pid=28085 next_prio=120 */
10) prog3-30194 | /* sched_wakeup: comm=kworker/u32:1
pid=29926 prio=120 target_cpu=010 */
10) prog3-30194 | 2.764 us | do_wait();
10) prog3-30194 | 0.285 us | tty_ldisc_ref_wait();
10) prog3-30194 | do_wait() {
0) prog3-30221 => kworker-17
10) prog3-30194 | /* sched_switch: prev_comm=prog3
prev_pid=30194 prev_prio=49 prev_state=S ==> next_comm=kworker/u32:1
next_pid=29926 next_prio=120 */
10) prog3-30194 => kworker-29926
15) prog3-30222 => kworker-28085
12) prog3-30220 | 1.051 us | tty_ldisc_ref_wait();
10) <idle>-0 | /* sched_wakeup: comm=prog3
pid=30194 prio=49 target_cpu=010 */
10) <idle>-0 | /* sched_switch: prev_comm=swapper/10
prev_pid=0 prev_prio=120 prev_state=R ==> next_comm=prog3 next_pid=30194
next_prio=49 */

```

```

12) prog3-30220 | | /* sched_switch: prev_comm=prog3
prev_pid=30220 prev_prio=49 prev_state=X ==> next_comm=swapper/12 next_pid=0
next_prio=120 */
10) <idle>-0 => prog3-30194
10) prog3-30194 | 732.893 us | } /* do_wait */
10) prog3-30194 | 0.381 us | tty_ldisc_ref_wait();
12) prog3-30220 => <idle>-0
10) prog3-30194 | | do_wait() {
10) prog3-30194 | | /* sched_switch: prev_comm=prog3
prev_pid=30194 prev_prio=49 prev_state=S ==> next_comm=swapper/10 next_pid=0
next_prio=120 */
9) prog3-30223 | 0.799 us | tty_ldisc_ref_wait();
10) prog3-30194 => <idle>-0
9) prog3-30223 | | /* sched_wakeup: comm=prog3 pid=30194
prio=49 target_cpu=010 */
9) prog3-30223 | | /* sched_switch: prev_comm=prog3
prev_pid=30223 prev_prio=49 prev_state=Z ==> next_comm=kworker/9:1
next_pid=23620 next_prio=120 */
10) QXcbEve-6642 | | /* sched_switch: prev_comm=QXcbEventQueue
prev_pid=6642 prev_prio=120 prev_state=S ==> next_comm=prog3 next_pid=30194
next_prio=49 */
10) QXcbEve-6642 => prog3-30194
10) prog3-30194 | 138.284 us | } /* do_wait */
10) prog3-30194 | 0.341 us | tty_ldisc_ref_wait();
10) prog3-30194 | | do_wait() {
10) prog3-30194 | | /* sched_switch: prev_comm=prog3
prev_pid=30194 prev_prio=49 prev_state=S ==> next_comm=swapper/10 next_pid=0
next_prio=120 */
9) prog3-30223 => kworker-23620
10) prog3-30194 => <idle>-0
13) prog3-30225 | 0.825 us | tty_ldisc_ref_wait();
13) prog3-30225 | | /* sched_switch: prev_comm=prog3
prev_pid=30225 prev_prio=49 prev_state=Z ==> next_comm=kworker/13:1
next_pid=28248 next_prio=120 */
10) <idle>-0 | | /* sched_wakeup: comm=prog3 pid=30194
prio=49 target_cpu=010 */
10) <idle>-0 | | /* sched_switch: prev_comm=swapper/10
prev_pid=0 prev_prio=120 prev_state=R ==> next_comm=prog3 next_pid=30194
next_prio=49 */
10) <idle>-0 => prog3-30194
10) prog3-30194 | 1517.741 us | } /* do_wait */
10) prog3-30194 | 0.273 us | tty_ldisc_ref_wait();
13) prog3-30225 => kworker-28248
10) prog3-30194 | | do_wait() {
10) prog3-30194 | | /* sched_switch: prev_comm=prog3
prev_pid=30194 prev_prio=49 prev_state=S ==> next_comm=swapper/10 next_pid=0
next_prio=120 */
10) prog3-30194 => <idle>-0
2) prog3-30226 | 0.963 us | tty_ldisc_ref_wait();
10) <idle>-0 | | /* sched_wakeup: comm=prog3 pid=30194
prio=49 target_cpu=010 */
10) <idle>-0 | | /* sched_switch: prev_comm=swapper/10
prev_pid=0 prev_prio=120 prev_state=R ==> next_comm=prog3 next_pid=30194
next_prio=49 */
2) prog3-30226 | | /* sched_switch: prev_comm=prog3
prev_pid=30226 prev_prio=49 prev_state=Z ==> next_comm=kworker/2:2
next_pid=25567 next_prio=120 */
10) <idle>-0 => prog3-30194
10) prog3-30194 | 3876.743 us | } /* do_wait */
10) prog3-30194 | 0.229 us | tty_ldisc_ref_wait();
10) prog3-30194 | 0.219 us | do_wait();

```

```

10) prog3-30194 | 0.146 us | tty_ldisc_ref_wait();
2) prog3-30226 => kworker-25567
10) prog3-30194 | /* sched_switch: prev_comm=prog3
prev_pid=30194 prev_prio=49 prev_state=Z ==> next_comm=swapper/10 next_pid=0
next_prio=120 */
10) prog3-30194 => <idle>-0
root@pitatir-Lenovo-Legion-Y9000P2021H:/sys/kernel/debug/tracing#

```

Из лога видно, что последним из дочерних процессов закончил работу процесс с PID 30226, об этом говорит строка со сменой контекста `/* sched_switch: prev_comm=prog3 prev_pid=30226 prev_prio=49 prev_state=Z ==> next_comm=kworker/2:2 next_pid=25567 next_prio=120 */`. Перед ним завершился процесс 30225 и 30223, что говорит о том, что вывод программы корректно отражает происходящее в ядре.

Стоит отметить, что процессы выполняются на разных ядрах произвольным образом, не так, как это задумано политикой FIFO. Несмотря на указание другого способа планирования, системой используется планировщик по умолчанию - `SCHED_OTHER`. Лог отражает конкуренцию процессов за процессорное время: много “перебивающих” смен контекста с помощью `sched_switch` говорят об этом.

Теперь сделаем так, чтобы та же программа запускалась на одном ядре: для этого заменим в скрипте строчку `/home/pitatir/LR7/prog3` на `sudo taskset --cpu-list 0 /home/pitatir/LR7/prog3`, которая ограничивает количество используемых ядер.

```

root@pitatir-Lenovo-Legion-Y9000P2021H:/sys/kernel/debug/tracing#
/home/pitatir/LR7/ftrace3.sh
kernel.ftrace_enabled = 1
Политика планирования родительского процесса: SCHED_FIFO

```

```

0) Дочерний процесс: pid = 31620 ppid = 31619
Checkpoint 0: процесс с pid = 31620
Процесс 0 с pid = 31620 закончен
1) Дочерний процесс: pid = 31621 ppid = 31619
Checkpoint 1: процесс с pid = 31621
Процесс 1 с pid = 31621 закончен
2) Дочерний процесс: pid = 31622 ppid = 31619
Checkpoint 2: процесс с pid = 31622
Процесс 2 с pid = 31622 закончен
3) Дочерний процесс: pid = 31623 ppid = 31619
Checkpoint 3: процесс с pid = 31623
Процесс 3 с pid = 31623 закончен
4) Дочерний процесс: pid = 31624 ppid = 31619

```

Checkpoint 4: процесс с pid = 31624
Процесс 4 с pid = 31624 закончен
5) Дочерний процесс: pid = 31625 ppid = 31619
Checkpoint 5: процесс с pid = 31625
Процесс 5 с pid = 31625 закончен
6) Дочерний процесс: pid = 31626 ppid = 31619
Checkpoint 6: процесс с pid = 31626
Процесс 6 с pid = 31626 закончен
7) Дочерний процесс: pid = 31627 ppid = 31619
Checkpoint 7: процесс с pid = 31627
Процесс 7 с pid = 31627 закончен
8) Дочерний процесс: pid = 31628 ppid = 31619
Checkpoint 8: процесс с pid = 31628
Процесс 8 с pid = 31628 закончен
9) Дочерний процесс: pid = 31629 ppid = 31619
Checkpoint 9: процесс с pid = 31629
Процесс 9 с pid = 31629 закончен
10) Дочерний процесс: pid = 31630 ppid = 31619
Checkpoint 10: процесс с pid = 31630
Процесс 10 с pid = 31630 закончен
11) Дочерний процесс: pid = 31631 ppid = 31619
Checkpoint 11: процесс с pid = 31631
Процесс 11 с pid = 31631 закончен
12) Дочерний процесс: pid = 31632 ppid = 31619
Checkpoint 12: процесс с pid = 31632
Процесс 12 с pid = 31632 закончен
13) Дочерний процесс: pid = 31633 ppid = 31619
Checkpoint 13: процесс с pid = 31633
Процесс 13 с pid = 31633 закончен
14) Дочерний процесс: pid = 31634 ppid = 31619
Checkpoint 14: процесс с pid = 31634
Процесс 14 с pid = 31634 закончен
15) Дочерний процесс: pid = 31635 ppid = 31619
Checkpoint 15: процесс с pid = 31635
Процесс 15 с pid = 31635 закончен
16) Дочерний процесс: pid = 31636 ppid = 31619
Checkpoint 16: процесс с pid = 31636
Процесс 16 с pid = 31636 закончен
17) Дочерний процесс: pid = 31637 ppid = 31619
Checkpoint 17: процесс с pid = 31637
Процесс 17 с pid = 31637 закончен
18) Дочерний процесс: pid = 31638 ppid = 31619
Checkpoint 18: процесс с pid = 31638
Процесс 18 с pid = 31638 закончен
19) Дочерний процесс: pid = 31639 ppid = 31619
Checkpoint 19: процесс с pid = 31639
Процесс 19 с pid = 31639 закончен
20) Дочерний процесс: pid = 31640 ppid = 31619
Checkpoint 20: процесс с pid = 31640
Процесс 20 с pid = 31640 закончен
21) Дочерний процесс: pid = 31641 ppid = 31619
Checkpoint 21: процесс с pid = 31641
Процесс 21 с pid = 31641 закончен
22) Дочерний процесс: pid = 31642 ppid = 31619
Checkpoint 22: процесс с pid = 31642
Процесс 22 с pid = 31642 закончен
23) Дочерний процесс: pid = 31643 ppid = 31619
Checkpoint 23: процесс с pid = 31643
Процесс 23 с pid = 31643 закончен
24) Дочерний процесс: pid = 31644 ppid = 31619
Checkpoint 24: процесс с pid = 31644


```

Процесс 24 с pid = 31644 закончен
25) Дочерний процесс: pid = 31645 ppid = 31619
Checkpoint 25: процесс с pid = 31645
Процесс 25 с pid = 31645 закончен
26) Дочерний процесс: pid = 31646 ppid = 31619
Checkpoint 26: процесс с pid = 31646
Процесс 26 с pid = 31646 закончен
27) Дочерний процесс: pid = 31647 ppid = 31619
Checkpoint 27: процесс с pid = 31647
Процесс 27 с pid = 31647 закончен
28) Дочерний процесс: pid = 31648 ppid = 31619
Checkpoint 28: процесс с pid = 31648
Процесс 28 с pid = 31648 закончен
29) Дочерний процесс: pid = 31649 ppid = 31619
Checkpoint 29: процесс с pid = 31649
Процесс 29 с pid = 31649 закончен
30) Дочерний процесс: pid = 31650 ppid = 31619
Checkpoint 30: процесс с pid = 31650
Процесс 30 с pid = 31650 закончен
31) Дочерний процесс: pid = 31651 ppid = 31619
Checkpoint 31: процесс с pid = 31651
Процесс 31 с pid = 31651 закончен
Процесс завершен: pid = 31620
Процесс завершен: pid = 31621
Процесс завершен: pid = 31622
Процесс завершен: pid = 31623
Процесс завершен: pid = 31624
Процесс завершен: pid = 31625
Процесс завершен: pid = 31626
Процесс завершен: pid = 31627
Процесс завершен: pid = 31628
Процесс завершен: pid = 31629
Процесс завершен: pid = 31630
Процесс завершен: pid = 31631
Процесс завершен: pid = 31632
Процесс завершен: pid = 31633
Процесс завершен: pid = 31634
Процесс завершен: pid = 31635
Процесс завершен: pid = 31636
Процесс завершен: pid = 31637
Процесс завершен: pid = 31638
Процесс завершен: pid = 31639
Процесс завершен: pid = 31640
Процесс завершен: pid = 31641
Процесс завершен: pid = 31642
Процесс завершен: pid = 31643
Процесс завершен: pid = 31644
Процесс завершен: pid = 31645
Процесс завершен: pid = 31646
Процесс завершен: pid = 31647
Процесс завершен: pid = 31648
Процесс завершен: pid = 31649
Процесс завершен: pid = 31650
Процесс завершен: pid = 31651
Завершение родительской программы pid = 31619
root@pitatir-Lenovo-Legion-Y9000P2021H:/sys/kernel/debug/tracing# cat trace
| grep -E 'prog3'
0) prog3-31620 | 0.141 us | __do_sys_getppid();
0) prog3-31620 | 0.117 us | tty_ldisc_ref_wait();
0) prog3-31620 | | /* sched_wakeup: comm=kworker/0:1
pid=17 prio=120 target_cpu=000 */

```

```

0) prog3-31620 | 0.766 us | tty_ldisc_ref_wait();
0) prog3-31620 | 0.154 us | tty_ldisc_ref_wait();
0) prog3-31620 | | /* sched_wakeup: comm=prog3 pid=31619
prio=49 target_cpu=000 */
0) prog3-31620 | | /* sched_switch: prev_comm=prog3
prev_pid=31620 prev_prio=49 prev_state=Z ==> next_comm=prog3 next_pid=31621
next_prio=49 */
0) prog3-31620 => prog3-31621
0) prog3-31621 | 0.283 us | __do_sys_getppid();
0) prog3-31621 | 0.109 us | tty_ldisc_ref_wait();
0) prog3-31621 | 0.237 us | tty_ldisc_ref_wait();
0) prog3-31621 | | /* sched_switch: prev_comm=prog3
prev_pid=31621 prev_prio=49 prev_state=R ==> next_comm=kworker/0:1
next_pid=17 next_prio=120 */
0) prog3-31621 => kworker-17
0) <idle>-0 | | /* sched_switch: prev_comm=swapper/0
prev_pid=0 prev_prio=120 prev_state=R ==> next_comm=prog3 next_pid=31621
next_prio=49 */
0) <idle>-0 => prog3-31621
0) prog3-31621 | 0.583 us | tty_ldisc_ref_wait();
0) prog3-31621 | | /* sched_switch: prev_comm=prog3
prev_pid=31621 prev_prio=49 prev_state=Z ==> next_comm=prog3 next_pid=31622
next_prio=49 */
0) prog3-31621 => prog3-31622
0) prog3-31622 | 0.382 us | __do_sys_getppid();
0) prog3-31622 | 0.152 us | tty_ldisc_ref_wait();
0) prog3-31622 | | /* sched_wakeup: comm=kworker/0:1
pid=17 prio=120 target_cpu=000 */
0) prog3-31622 | | stop_one_cpu_nowait() {
0) prog3-31622 | | /* sched_wakeup: comm=migration/0
pid=15 prio=0 target_cpu=000 */
0) prog3-31622 | 1.515 us | }
0) prog3-31622 | | /* sched_switch: prev_comm=prog3
prev_pid=31622 prev_prio=49 prev_state=R ==> next_comm=migration/0
next_pid=15 next_prio=0 */
0) prog3-31622 => migrati-15
0) migrati-15 | | /* sched_switch: prev_comm=migration/0
prev_pid=15 prev_prio=0 prev_state=S ==> next_comm=prog3 next_pid=31622
next_prio=49 */
0) migrati-15 => prog3-31622
0) prog3-31622 | 0.131 us | tty_ldisc_ref_wait();
0) prog3-31622 | 0.778 us | tty_ldisc_ref_wait();
0) prog3-31622 | | /* sched_switch: prev_comm=prog3
prev_pid=31622 prev_prio=49 prev_state=Z ==> next_comm=prog3 next_pid=31623
next_prio=49 */
0) prog3-31622 => prog3-31623
0) prog3-31623 | 0.356 us | __do_sys_getppid();
0) prog3-31623 | 0.103 us | tty_ldisc_ref_wait();
0) prog3-31623 | 0.638 us | tty_ldisc_ref_wait();
0) prog3-31623 | | /* sched_wakeup: comm=ksoftirqd/0
pid=13 prio=120 target_cpu=000 */
0) prog3-31623 | | /* sched_switch: prev_comm=prog3
prev_pid=31623 prev_prio=49 prev_state=R ==> next_comm=ksoftirqd/0
next_pid=13 next_prio=120 */
0) prog3-31623 => ksoftir-13
0) <idle>-0 | | /* sched_switch: prev_comm=swapper/0
prev_pid=0 prev_prio=120 prev_state=R ==> next_comm=prog3 next_pid=31623
next_prio=49 */
0) <idle>-0 => prog3-31623
0) prog3-31623 | | /* sched_wakeup: comm=kworker/0:1
pid=17 prio=120 target_cpu=000 */

```

```

0) prog3-31623 | 1.071 us | tty_ldisc_ref_wait();
0) prog3-31623 | | /* sched_switch: prev_comm=prog3
prev_pid=31623 prev_prio=49 prev_state=Z ==> next_comm=prog3 next_pid=31624
next_prio=49 */
0) prog3-31623 => prog3-31624
0) prog3-31624 | 0.291 us | __do_sys_getppid();
0) prog3-31624 | 0.134 us | tty_ldisc_ref_wait();
0) prog3-31624 | 0.940 us | tty_ldisc_ref_wait();
0) prog3-31624 | 0.760 us | tty_ldisc_ref_wait();
0) prog3-31624 | | /* sched_switch: prev_comm=prog3
prev_pid=31624 prev_prio=49 prev_state=Z ==> next_comm=prog3 next_pid=31625
next_prio=49 */
0) prog3-31624 => prog3-31625

```

Вывод в терминал и трассировка ядра также совпадают, но в этот раз используется 2 ядра: №0 и №1, и планировщик FIFO корректно выполняет свою работу. Конкуренции между процессами в данной ситуации нет, потому что они по очереди получают доступ к процессору, но можно судить о занимаемом разными процессами процессорном времени (столбец DURATION). Например, дочерний процесс с PID 31620 занял 1,178 мсек, а процесс с PID 31624 - 2,115 мсек.

Повторим запуск программы на одном ядре, но уже с политикой SCHED_RR.

```

root@pitatir-Lenovo-Legion-Y9000P2021H:/sys/kernel/debug/tracing#
/home/pitatir/LR7/ftrace3.sh
kernel.ftrace_enabled = 1
Политика планирования родительского процесса: SCHED_RR

```

```

0) Дочерний процесс: pid = 32138 ppid = 32137
1) Дочерний процесс: pid = 32139 ppid = 32137
2) Дочерний процесс: pid = 32140 ppid = 32137
3) Дочерний процесс: pid = 32141 ppid = 32137
4) Дочерний процесс: pid = 32142 ppid = 32137
5) Дочерний процесс: pid = 32143 ppid = 32137
6) Дочерний процесс: pid = 32144 ppid = 32137
7) Дочерний процесс: pid = 32145 ppid = 32137
8) Дочерний процесс: pid = 32146 ppid = 32137
9) Дочерний процесс: pid = 32147 ppid = 32137
10) Дочерний процесс: pid = 32148 ppid = 32137
11) Дочерний процесс: pid = 32149 ppid = 32137
12) Дочерний процесс: pid = 32150 ppid = 32137
13) Дочерний процесс: pid = 32151 ppid = 32137
14) Дочерний процесс: pid = 32152 ppid = 32137
15) Дочерний процесс: pid = 32153 ppid = 32137
16) Дочерний процесс: pid = 32154 ppid = 32137
17) Дочерний процесс: pid = 32155 ppid = 32137
18) Дочерний процесс: pid = 32156 ppid = 32137
19) Дочерний процесс: pid = 32157 ppid = 32137

```

20) Дочерний процесс: pid = 32158 ppid = 32137
21) Дочерний процесс: pid = 32159 ppid = 32137
22) Дочерний процесс: pid = 32160 ppid = 32137
23) Дочерний процесс: pid = 32161 ppid = 32137
24) Дочерний процесс: pid = 32162 ppid = 32137
25) Дочерний процесс: pid = 32163 ppid = 32137
26) Дочерний процесс: pid = 32164 ppid = 32137
27) Дочерний процесс: pid = 32165 ppid = 32137
28) Дочерний процесс: pid = 32166 ppid = 32137
29) Дочерний процесс: pid = 32167 ppid = 32137
30) Дочерний процесс: pid = 32168 ppid = 32137
31) Дочерний процесс: pid = 32169 ppid = 32137
Checkpoint 0: процесс с pid = 32138
Checkpoint 1: процесс с pid = 32139
Checkpoint 2: процесс с pid = 32140
Checkpoint 3: процесс с pid = 32141
Checkpoint 4: процесс с pid = 32142
Checkpoint 5: процесс с pid = 32143
Checkpoint 6: процесс с pid = 32144
Checkpoint 7: процесс с pid = 32145
Checkpoint 8: процесс с pid = 32146
Checkpoint 9: процесс с pid = 32147
Checkpoint 10: процесс с pid = 32148
Checkpoint 11: процесс с pid = 32149
Checkpoint 12: процесс с pid = 32150
Checkpoint 13: процесс с pid = 32151
Checkpoint 14: процесс с pid = 32152
Checkpoint 15: процесс с pid = 32153
Checkpoint 16: процесс с pid = 32154
Checkpoint 17: процесс с pid = 32155
Checkpoint 18: процесс с pid = 32156
Checkpoint 19: процесс с pid = 32157
Checkpoint 20: процесс с pid = 32158
Checkpoint 21: процесс с pid = 32159
Checkpoint 22: процесс с pid = 32160
Checkpoint 23: процесс с pid = 32161
Checkpoint 24: процесс с pid = 32162
Checkpoint 25: процесс с pid = 32163
Checkpoint 26: процесс с pid = 32164
Checkpoint 27: процесс с pid = 32165
Checkpoint 28: процесс с pid = 32166
Checkpoint 29: процесс с pid = 32167
Checkpoint 30: процесс с pid = 32168
Checkpoint 31: процесс с pid = 32169
Процесс 0 с pid = 32138 закончен
Процесс 1 с pid = 32139 закончен
Процесс 2 с pid = 32140 закончен
Процесс 3 с pid = 32141 закончен
Процесс 4 с pid = 32142 закончен
Процесс 5 с pid = 32143 закончен
Процесс 6 с pid = 32144 закончен
Процесс 7 с pid = 32145 закончен
Процесс 8 с pid = 32146 закончен
Процесс 9 с pid = 32147 закончен
Процесс 10 с pid = 32148 закончен
Процесс 11 с pid = 32149 закончен
Процесс 12 с pid = 32150 закончен
Процесс 13 с pid = 32151 закончен
Процесс 14 с pid = 32152 закончен
Процесс 15 с pid = 32153 закончен
Процесс 16 с pid = 32154 закончен

Процесс 17 с pid = 32155 закончен
Процесс 18 с pid = 32156 закончен
Процесс 19 с pid = 32157 закончен
Процесс 20 с pid = 32158 закончен
Процесс 21 с pid = 32159 закончен
Процесс 22 с pid = 32160 закончен
Процесс 23 с pid = 32161 закончен
Процесс 24 с pid = 32162 закончен
Процесс 25 с pid = 32163 закончен
Процесс 26 с pid = 32164 закончен
Процесс 27 с pid = 32165 закончен
Процесс 28 с pid = 32166 закончен
Процесс 29 с pid = 32167 закончен
Процесс 30 с pid = 32168 закончен
Процесс 31 с pid = 32169 закончен

Процесс завершен: pid = 32138
Процесс завершен: pid = 32139
Процесс завершен: pid = 32140
Процесс завершен: pid = 32141
Процесс завершен: pid = 32142
Процесс завершен: pid = 32143
Процесс завершен: pid = 32144
Процесс завершен: pid = 32145
Процесс завершен: pid = 32146
Процесс завершен: pid = 32147
Процесс завершен: pid = 32148
Процесс завершен: pid = 32149
Процесс завершен: pid = 32150
Процесс завершен: pid = 32151
Процесс завершен: pid = 32152
Процесс завершен: pid = 32153
Процесс завершен: pid = 32154
Процесс завершен: pid = 32155
Процесс завершен: pid = 32156
Процесс завершен: pid = 32157
Процесс завершен: pid = 32158
Процесс завершен: pid = 32159
Процесс завершен: pid = 32160
Процесс завершен: pid = 32161
Процесс завершен: pid = 32162
Процесс завершен: pid = 32163
Процесс завершен: pid = 32164
Процесс завершен: pid = 32165
Процесс завершен: pid = 32166
Процесс завершен: pid = 32167
Процесс завершен: pid = 32168
Процесс завершен: pid = 32169

Завершение родительской программы pid = 32137

root@pitatir-Lenovo-Legion-Y9000P2021H:/sys/kernel/debug/tracing#

root@pitatir-Lenovo-Legion-Y9000P2021H:/sys/kernel/debug/tracing# cat trace

| grep -E 'prog3'

0) prog3-32138 | 0.179 us | __do_sys_getppid();
0) prog3-32138 | 0.132 us | tty_ldisc_ref_wait();
0) prog3-32138 | | /* sched_switch: prev_comm=prog3
prev_pid=32138 prev_prio=49 prev_state=R ==> next_comm=prog3 next_pid=32139
next_prio=49 */

0) prog3-32138 => prog3-32139
0) prog3-32139 | 0.278 us | __do_sys_getppid();
0) prog3-32139 | 0.192 us | tty_ldisc_ref_wait();

```

0) prog3-32139 | | /* sched_switch: prev_comm=prog3
prev_pid=32139 prev_prio=49 prev_state=R ==> next_comm=prog3 next_pid=32140
next_prio=49 */
0) prog3-32139 => prog3-32140
0) prog3-32140 | 0.383 us | __do_sys_getppid();
0) prog3-32140 | 0.202 us | tty_ldisc_ref_wait();
0) prog3-32140 | | /* sched_switch: prev_comm=prog3
prev_pid=32140 prev_prio=49 prev_state=R ==> next_comm=prog3 next_pid=32141
next_prio=49 */
0) prog3-32140 => prog3-32141
0) prog3-32141 | 0.323 us | __do_sys_getppid();
0) prog3-32141 | 0.200 us | tty_ldisc_ref_wait();
0) prog3-32141 | | /* sched_wakeup: comm=kworker/0:1
pid=17 prio=120 target_cpu=000 */
0) prog3-32141 | | /* sched_wakeup: comm=yandex_browser
pid=21146 prio=120 target_cpu=000 */
0) prog3-32141 | | /* sched_wakeup: comm=yandex_browser
pid=12882 prio=120 target_cpu=000 */
0) prog3-32141 | | /* sched_wakeup: comm=yandex_browser
pid=3949 prio=120 target_cpu=000 */
0) prog3-32141 | | /* sched_wakeup: comm=yandex_browser
pid=4000 prio=120 target_cpu=000 */
0) prog3-32141 | | /* sched_wakeup: comm=yandex_browser
pid=12548 prio=120 target_cpu=000 */
0) prog3-32141 | | /* sched_switch: prev_comm=prog3
prev_pid=32141 prev_prio=49 prev_state=R ==> next_comm=prog3 next_pid=32142
next_prio=49 */
0) prog3-32141 => prog3-32142
0) prog3-32142 | 0.971 us | __do_sys_getppid();
0) prog3-32142 | 0.345 us | tty_ldisc_ref_wait();
0) prog3-32142 | | /* sched_switch: prev_comm=prog3
prev_pid=32142 prev_prio=49 prev_state=R ==> next_comm=prog3 next_pid=32143
next_prio=49 */
0) prog3-32142 => prog3-32143

```

Вывод в терминал и трассировка ядра совпадают, используется только 1 ядра №0, и планировщик RR корректно выполняет свою работу. Конкуренция между процессами в данной ситуации заключается в последовательном выполнении друг за другом своих действий в течение такта, однако время, занимаемое у процессора тем или иным процессом по-прежнему различается: 0,311 мсек у процесса с PID 32138 и 0,523 мсек у процесса с PID 32141, например.

При политике планирования OTHER на одном ядре конкуренция видна лучше всего: лог получается в разы длиннее, чем при других способах планирования, и его большую часть занимают смены контекста с одного процесса на другой.

```

root@pitatir-Lenovo-Legion-Y9000P2021H:/sys/kernel/debug/tracing#
/home/pitatir/LR7/ftrace3.sh

```

kernel.ftrace_enabled = 1

Политика планирования родительского процесса: SCHED_OTHER

6) Дочерний процесс: pid = 32446 ppid = 32439
7) Дочерний процесс: pid = 32447 ppid = 32439
8) Дочерний процесс: pid = 32448 ppid = 32439
9) Дочерний процесс: pid = 32449 ppid = 32439
10) Дочерний процесс: pid = 32450 ppid = 32439
11) Дочерний процесс: pid = 32451 ppid = 32439
12) Дочерний процесс: pid = 32452 ppid = 32439
13) Дочерний процесс: pid = 32453 ppid = 32439
14) Дочерний процесс: pid = 32454 ppid = 32439
15) Дочерний процесс: pid = 32455 ppid = 32439
16) Дочерний процесс: pid = 32456 ppid = 32439
17) Дочерний процесс: pid = 32457 ppid = 32439
18) Дочерний процесс: pid = 32458 ppid = 32439
19) Дочерний процесс: pid = 32459 ppid = 32439
20) Дочерний процесс: pid = 32460 ppid = 32439
21) Дочерний процесс: pid = 32461 ppid = 32439
5) Дочерний процесс: pid = 32445 ppid = 32439
22) Дочерний процесс: pid = 32462 ppid = 32439
23) Дочерний процесс: pid = 32463 ppid = 32439
24) Дочерний процесс: pid = 32464 ppid = 32439
25) Дочерний процесс: pid = 32465 ppid = 32439
26) Дочерний процесс: pid = 32466 ppid = 32439
27) Дочерний процесс: pid = 32467 ppid = 32439
28) Дочерний процесс: pid = 32468 ppid = 32439
29) Дочерний процесс: pid = 32469 ppid = 32439
30) Дочерний процесс: pid = 32470 ppid = 32439
31) Дочерний процесс: pid = 32471 ppid = 32439
4) Дочерний процесс: pid = 32444 ppid = 32439
3) Дочерний процесс: pid = 32443 ppid = 32439
2) Дочерний процесс: pid = 32442 ppid = 32439
1) Дочерний процесс: pid = 32441 ppid = 32439
0) Дочерний процесс: pid = 32440 ppid = 32439
Checkpoint 3: процесс с pid = 32443
Checkpoint 6: процесс с pid = 32446
Checkpoint 26: процесс с pid = 32466
Checkpoint 7: процесс с pid = 32447
Checkpoint 9: процесс с pid = 32449
Checkpoint 13: процесс с pid = 32453
Checkpoint 15: процесс с pid = 32455
Checkpoint 16: процесс с pid = 32456
Checkpoint 17: процесс с pid = 32457
Checkpoint 18: процесс с pid = 32458
Checkpoint 19: процесс с pid = 32459
Checkpoint 20: процесс с pid = 32460
Checkpoint 21: процесс с pid = 32461
Checkpoint 22: процесс с pid = 32462
Checkpoint 24: процесс с pid = 32464
Checkpoint 23: процесс с pid = 32463
Checkpoint 28: процесс с pid = 32468
Checkpoint 29: процесс с pid = 32469
Checkpoint 30: процесс с pid = 32470
Checkpoint 31: процесс с pid = 32471
Checkpoint 4: процесс с pid = 32444
Checkpoint 2: процесс с pid = 32442
Checkpoint 8: процесс с pid = 32448
Checkpoint 10: процесс с pid = 32450
Checkpoint 11: процесс с pid = 32451
Checkpoint 12: процесс с pid = 32452

Checkpoint 14: процесс с pid = 32454
Checkpoint 25: процесс с pid = 32465
Checkpoint 27: процесс с pid = 32467
Checkpoint 0: процесс с pid = 32440
Checkpoint 1: процесс с pid = 32441
Checkpoint 5: процесс с pid = 32445
Процесс 22 с pid = 32462 закончен
Процесс завершен: pid = 32462
Процесс 31 с pid = 32471 закончен
Процесс завершен: pid = 32471
Процесс 7 с pid = 32447 закончен
Процесс завершен: pid = 32447
Процесс 11 с pid = 32451 закончен
Процесс завершен: pid = 32451
Процесс 13 с pid = 32453 закончен
Процесс завершен: pid = 32453
Процесс 16 с pid = 32456 закончен
Процесс завершен: pid = 32456
Процесс 15 с pid = 32455 закончен
Процесс завершен: pid = 32455
Процесс 17 с pid = 32457 закончен
Процесс завершен: pid = 32457
Процесс 21 с pid = 32461 закончен
Процесс завершен: pid = 32461
Процесс 24 с pid = 32464 закончен
Процесс завершен: pid = 32464
Процесс 27 с pid = 32467 закончен
Процесс завершен: pid = 32467
Процесс 30 с pid = 32470 закончен
Процесс завершен: pid = 32470
Процесс 28 с pid = 32468 закончен
Процесс завершен: pid = 32468
Процесс 9 с pid = 32449 закончен
Процесс завершен: pid = 32449
Процесс 18 с pid = 32458 закончен
Процесс завершен: pid = 32458
Процесс 4 с pid = 32444 закончен
Процесс завершен: pid = 32444
Процесс 10 с pid = 32450 закончен
Процесс завершен: pid = 32450
Процесс 0 с pid = 32440 закончен
Процесс завершен: pid = 32440
Процесс 19 с pid = 32459 закончен
Процесс завершен: pid = 32459
Процесс 6 с pid = 32446 закончен
Процесс завершен: pid = 32446
Процесс 26 с pid = 32466 закончен
Процесс завершен: pid = 32466
Процесс 20 с pid = 32460 закончен
Процесс завершен: pid = 32460
Процесс 25 с pid = 32465 закончен
Процесс завершен: pid = 32465
Процесс 12 с pid = 32452 закончен
Процесс завершен: pid = 32452
Процесс 14 с pid = 32454 закончен
Процесс завершен: pid = 32454
Процесс 29 с pid = 32469 закончен
Процесс завершен: pid = 32469
Процесс 23 с pid = 32463 закончен
Процесс завершен: pid = 32463
Процесс 3 с pid = 32443 закончен

Процесс завершен: pid = 32443
Процесс 2 с pid = 32442 закончен
Процесс завершен: pid = 32442
Процесс 5 с pid = 32445 закончен
Процесс завершен: pid = 32445
Процесс 8 с pid = 32448 закончен
Процесс завершен: pid = 32448
Процесс 1 с pid = 32441 закончен
Процесс завершен: pid = 32441
Завершение родительской программы pid = 32439

```
0) prog3-32446 => prog3-32447
0) prog3-32447 | /* sched_switch: prev_comm=prog3
prev_pid=32447 prev_prio=120 prev_state=R ==> next_comm=prog3 next_pid=32448
next_prio=120 */
0) prog3-32447 => prog3-32448
0) prog3-32448 | /* sched_switch: prev_comm=prog3
prev_pid=32448 prev_prio=120 prev_state=R ==> next_comm=prog3 next_pid=32449
next_prio=120 */
0) prog3-32448 => prog3-32449
0) prog3-32449 | /* sched_switch: prev_comm=prog3
prev_pid=32449 prev_prio=120 prev_state=R ==> next_comm=prog3 next_pid=32450
next_prio=120 */
0) prog3-32449 => prog3-32450
0) prog3-32450 | /* sched_switch: prev_comm=prog3
prev_pid=32450 prev_prio=120 prev_state=R ==> next_comm=prog3 next_pid=32451
next_prio=120 */
0) prog3-32450 => prog3-32451
0) prog3-32451 | /* sched_switch: prev_comm=prog3
prev_pid=32451 prev_prio=120 prev_state=R ==> next_comm=prog3 next_pid=32452
next_prio=120 */
0) prog3-32451 => prog3-32452
0) prog3-32452 | /* sched_switch: prev_comm=prog3
prev_pid=32452 prev_prio=120 prev_state=R ==> next_comm=prog3 next_pid=32453
next_prio=120 */
0) prog3-32452 => prog3-32453
0) prog3-32453 | /* sched_switch: prev_comm=prog3
prev_pid=32453 prev_prio=120 prev_state=R ==> next_comm=prog3 next_pid=32454
next_prio=120 */
0) prog3-32453 => prog3-32454
0) prog3-32454 | /* sched_switch: prev_comm=prog3
prev_pid=32454 prev_prio=120 prev_state=R ==> next_comm=prog3 next_pid=32455
next_prio=120 */
0) prog3-32454 => prog3-32455
0) prog3-32455 | /* sched_switch: prev_comm=prog3
prev_pid=32455 prev_prio=120 prev_state=R ==> next_comm=prog3 next_pid=32456
next_prio=120 */
0) prog3-32455 => prog3-32456
0) prog3-32456 | /* sched_switch: prev_comm=prog3
prev_pid=32456 prev_prio=120 prev_state=R ==> next_comm=prog3 next_pid=32457
next_prio=120 */
0) prog3-32456 => prog3-32457
0) prog3-32457 | /* sched_switch: prev_comm=prog3
prev_pid=32457 prev_prio=120 prev_state=R ==> next_comm=prog3 next_pid=32458
next_prio=120 */
```

Вывод.

Изучено профилирование, способы трассировки ядра. Знания успешно применены их на практике. Динамика выполнения и событий проанализированы.